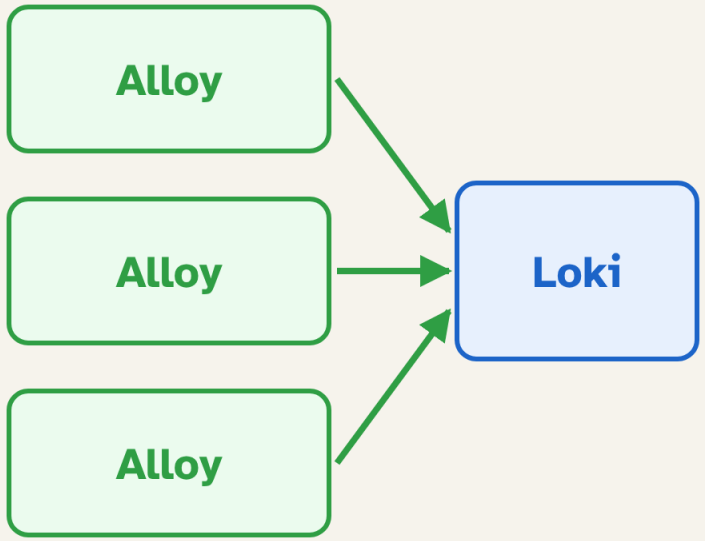
LEZIONE 4

Log centralizzati

ITS CyberSecurity - Jesi (AN) 2026/2027

Corso Logging, Monitoring, Troubleshooting

prof. Luca Ferroni



tanti PC, un solo posto

Dove eravamo

Lezione 3: Docker fino al primo container

1. `docker run hello-world` — il primo container
2. `docker ps -a` — anche quelli fermi
3. `-p 8007:80` — porta fuori : porta dentro

Oggi: dal comando al file compose.yml,

e poi i log di tutta la classe in un posto solo.

Da docker run a compose.yml

```
$ docker run -d --name web -p 8080:80 nginx:alpine
```

compose.yml

```yaml
services:
  web:
    image: nginx:alpine
    ports:
      - "8080:80"
```

Stesso container, scritto in un file.

Si legge, si copia, si versiona con git.

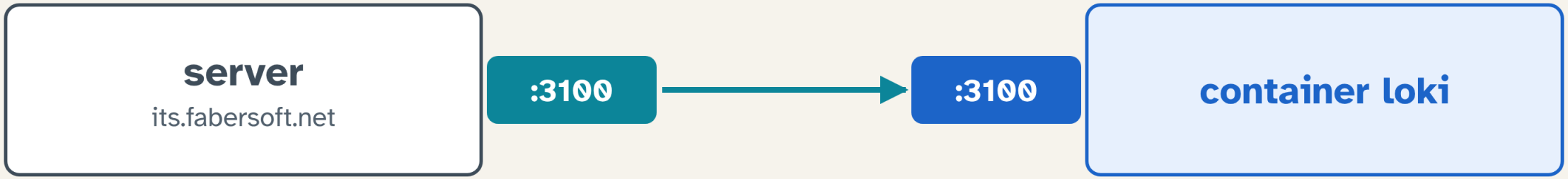
YAML: spazi, mai tab.

ports: fuori e dentro

La stessa regola di docker run -p

```
"3100:3100"
```

porta del server porta del container



Chi arriva da fuori bussa alla porta del server: Docker lo passa al container.

volumes: i dati stanno fuori

Due modi di mettere qualcosa dentro un container

```
loki-data:/loki
```

Un disco gestito da Docker.

Butti via il container, i log restano.

```
./loki/config.yml:/etc/loki/config.yml:ro
```

Un file della tua cartella, visto dentro il container.

:ro = sola lettura: il container lo legge, non lo cambia.

restart: unless-stopped

Il caso della lezione 1: Grafana spento, nessuno lo riaccende

```
restart: unless-stopped
```

- Si ferma da solo o si riavvia il server: riparte.
- L'hai fermato tu: resta fermo.

Lo mettiamo in tutti i servizi, server e client.

I comandi di docker compose

Sempre dalla cartella dove c'è il compose.yml

1. `docker compose up -d` — avvia tutto
2. `docker compose ps` — chi gira?
3. `docker compose logs -f alloy` — log in tempo reale
4. `docker compose down` — ferma e cancella i container

I volumi restano: down non cancella i dati.

docker run o docker compose up?

`docker run`

un container

tutto scritto
sulla riga di comando

lo rilanci: un altro
container (o un errore)

`docker compose up`

tutti i servizi del file

tutto scritto
in compose.yml

lo rilanci: cambia solo
quello che hai modificato

Dove finiscono i log sulla macchina

Quattro posti. I log vanno conservati a norma di legge, anche fuori dalla macchina

systemd-journald

binario, si legge solo con journalctl
è il default delle distribuzioni recenti

```
$ journalctl -u ssh -f
```

syslog (rsyslog)

testo, in /var/log/syslog e /var/log/auth.log
si legge con cat, grep, tail

```
$ tail -f /var/log/auth.log
```

file dell'applicazione

in /var/log o dove decide l'applicazione
es. Apache, Nginx, Postfix

```
$ tail -f /var/log/apache2/access.log
```

log di Docker

stdout e stderr del container
2>&1 per passarli entrambi a grep

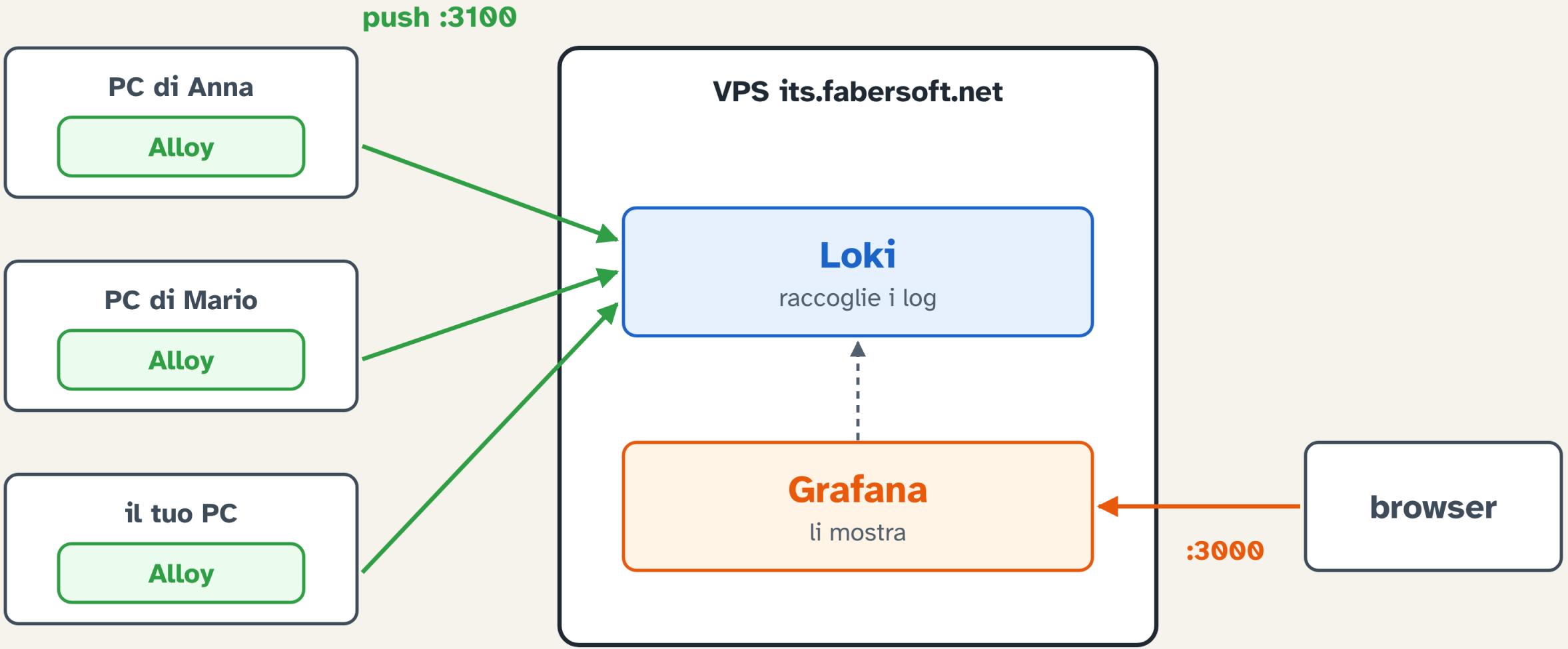
```
$ docker logs -f grafana 2>&1 | grep -i error
```

Come si centralizzano i log

Quattro architetture: nel corso usiamo Loki

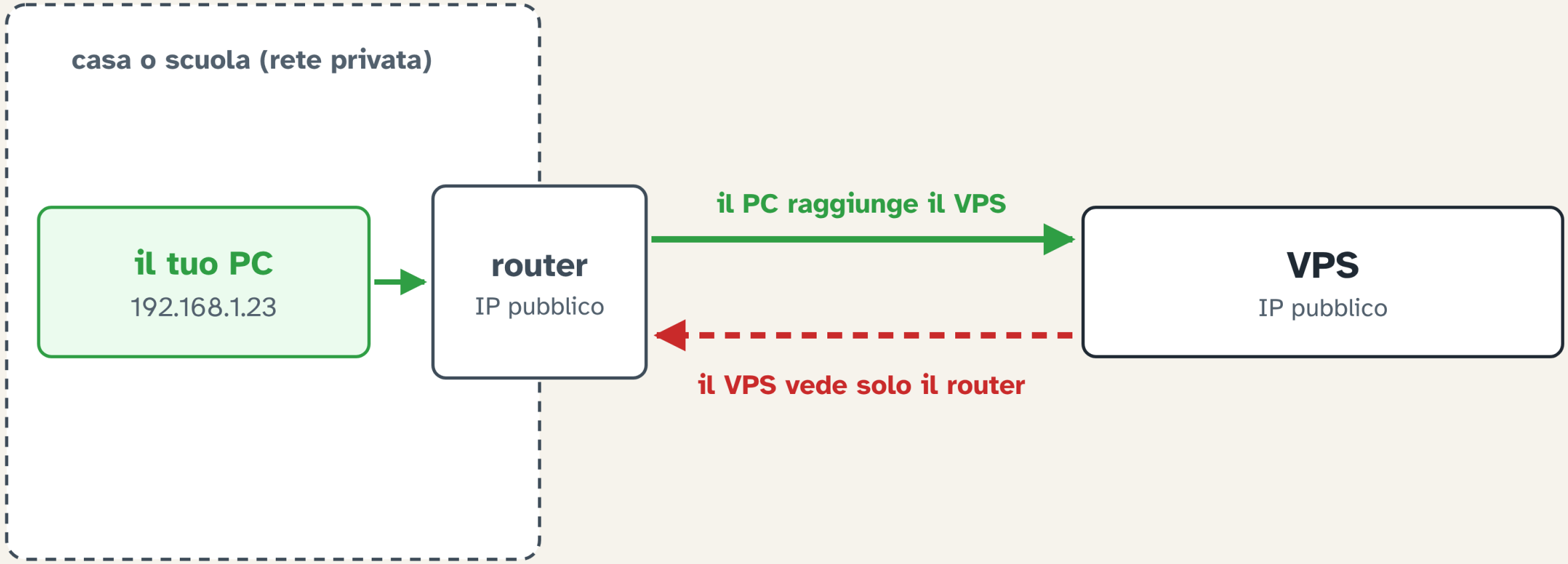
	raccolta	storage	visualizzazione	nota
syslog	inoltro al server rsyslog UDP/TCP 514, TLS 6514	file di testo	grep, tail	la più semplice: c'è già ovunque
ELK Elastic Stack	Beats o Logstash	Elasticsearch, indice full-text	Kibana	ricerche potenti, ma pesante in RAM e disco
Graylog	GELF, syslog, Beats	OpenSearch (più MongoDB)	web di Graylog	pronto all'uso, con stream e allarmi
Loki nel corso	Alloy	indicizza solo le etichette	Grafana	leggero: per questo lo usiamo

Cosa costruiamo



Perché è il PC a spedire

Livello 3: chi riesce a raggiungere chi?



Quindi è Alloy, sul PC, ad aprire la connessione e spedire.

Il server (1/2): Loki

logging/vps-corso/compose.yml

```
services:
  loki:
    image: grafana/loki:3.7.0
    command: -config.file=/etc/loki/config.yml
    volumes:
      - ./loki/config.yml:/etc/loki/config.yml:ro
      - loki-data:/loki
    ports:
      - "3100:3100"
    restart: unless-stopped
```

← immagine e versione

← il file di configurazione

← configurazione e dati

← dove arrivano i log

← riparte da solo

Il server (2/2): Grafana

logging/vps-corso/compose.yml, continua

```yaml
  grafana:
    image: grafana/grafana:13.2.4
    volumes:
      - grafana-data:/var/lib/grafana
    ports:
      - "3000:3000"
    restart: unless-stopped


volumes:
  loki-data:
  grafana-data:
```

← utenti e cruscotti

← qui apri il browser

← i volumi di Docker, dichiarati in fondo

Il server: lo avviamo insieme

Sul VPS, lo faccio io: voi controllate dal vostro PC

1. `cd its-jesi-lessons/logging/vps-corso`
2. `docker compose up -d`
3. `docker compose ps` — tutti e due Up?
4. `curl http://its.fabersoft.net:3100/ready` — ready
5. `http://its.fabersoft.net:3000` — admin / admin
6. `Sorgente dati Loki: http://loki:3100`

Il client: Alloy sul tuo PC

logging/pc-studente/compose.yml

```yaml
services:
  alloy:
    image: grafana/alloy:v1.20.1
    hostname: mario-rossi
    command:
      - run
      - --server.http.listen-addr=0.0.0.0:12345
      - --storage.path=/var/lib/alloy/data
      - /etc/alloy/config.alloy
    ports:
      - "127.0.0.1:12345:12345"
    volumes:
      - ./alloy/config.alloy:/etc/alloy/config.alloy:ro
      - ..:/log-corso:ro
      - alloy-data:/var/lib/alloy/data
    restart: unless-stopped
```

← il TUO nome

← avvia Alloy con il suo file di configurazione

← pagina di Alloy, solo dal PC

← la configurazione

← i .log della lezione 2

← dove è arrivato a leggere

Il client: config.alloy

logging/pc-studente/alloy/config.alloy

```
local.file_match "corso" {
  path_targets = [{"__path__" = "/log-corso/*.log", job = "corso"}]
}

loki.source.file "corso" {
  targets    = local.file_match.corso.targets
  forward_to = [loki.write.vps.receiver]
}

loki.write "vps" {
  endpoint {
    url = "http://its.fabersoft.net:3100/loki/api/v1/push"
  }
  external_labels = { host = constants.hostname }
}
```

1. trova i file

2. leggili

3. spediscili

a Loki, con l'etichetta host

Una sola riga da cambiare

In pc-studente/compose.yml

```
hostname: mario-rossi
```



in Grafana diventa

```
{host="mario-rossi"}
```

Nome e cognome, minuscolo, con il trattino. Due studenti, due nomi diversi.

Il client: avvio e verifica

Sul tuo PC

1. `cd its-jesi-lessons/logging/pc-studente`
2. `docker compose up -d`
3. `docker compose logs -f alloy` — errori?
4. `http://localhost:12345` — tutto verde?

Se docker compose up resta fermo a scaricare: l'immagine si scarica da casa.

Grafana: Explore

http://its.fabersoft.net:3000

1. **Menu → Explore**
2. **Sorgente dati: Loki**
3. `{host="mario-rossi"}` il tuo nome
4. **Run query**

Le righe di sshd_classic.log, apache_access.log e mail.log: le stesse della lezione 2.

LogQL è il vostro grep

A sinistra la lezione 2, a destra Grafana

Shell

```
grep "Failed password" \
  sshd_classic.log
```

Grafana (LogQL)

```
{filename="/log-corso/sshd_classic.log"}
  |= "Failed password"
```

```
grep -v "Accepted" \
  sshd_classic.log
```

```
{filename="/log-corso/sshd_classic.log"}
  != "Accepted"
```

|= tiene le righe che contengono il testo, != le toglie.

Stesso numero

Quante righe parlano dell'IP 45.9.14.207?

```
$ cat *.log | grep -c 45.9.14.207
79
```

```
sum(count_over_time({host="mario-rossi"} |= "45.9.14.207" [1h]))
79
```

Stessi log, stessa domanda, stessa risposta: ma ora sono tutti in un posto.

Tutta la classe in un posto

Una sola domanda, una riga per ogni studente

```
sum by (host) (count_over_time({job="corso"} [1h]))
```

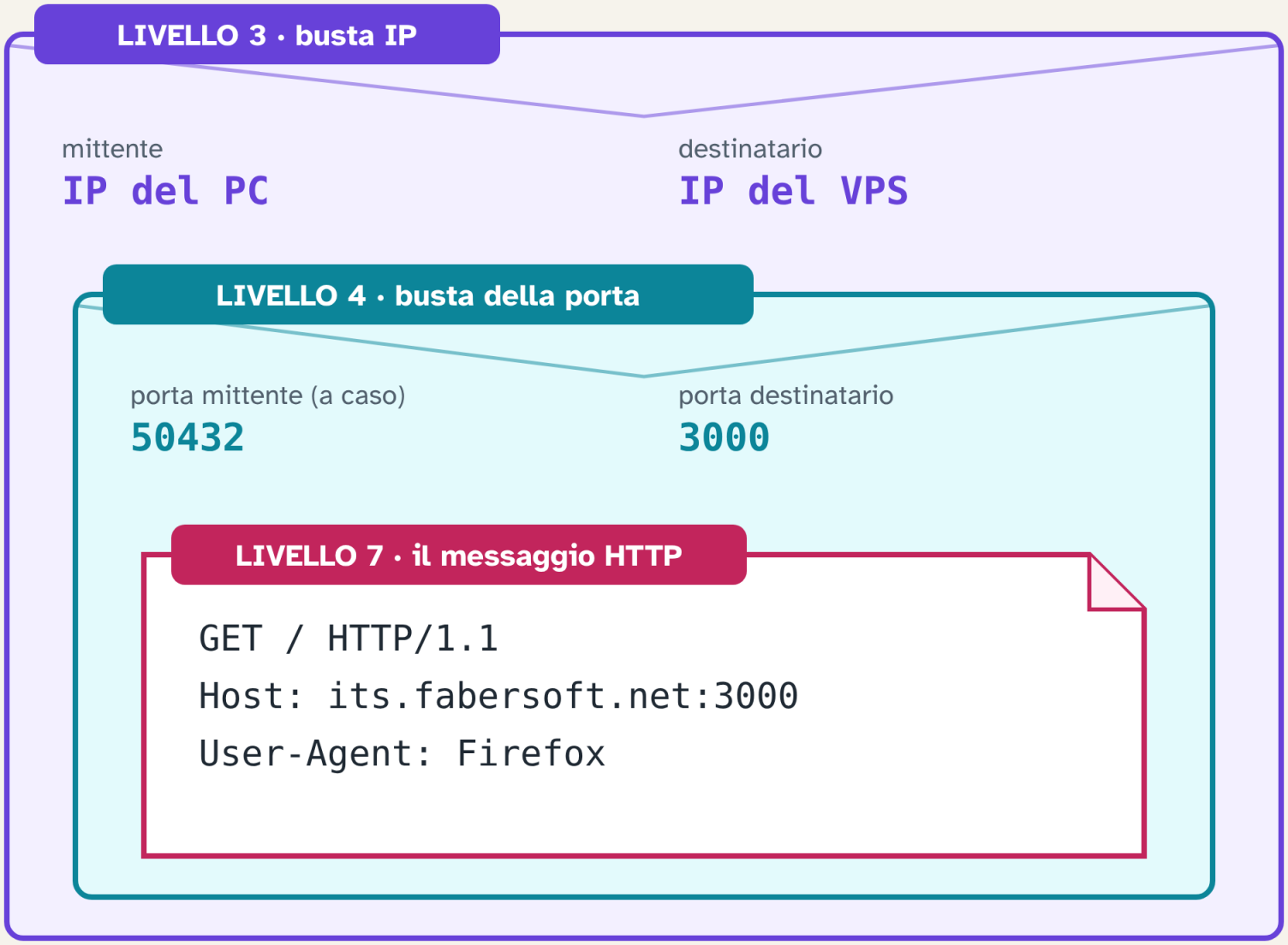
anna-bianchi	282
luca-verdi	282
mario-rossi	282
...	

Questo vuol dire centralizzato.

282 = le righe dei 4 file

Incapsulamento: tre buste

Il browser chiede una pagina a Grafana sul VPS



Chi apre quale busta

3 · indirizzo IP — `ping`

I router lungo Internet: portano il pacchetto al VPS.

4 · porta — `nc -zv`

Il sistema operativo del VPS: consegna al programma in ascolto.

7 · messaggio — `curl`

Solo il programma, qui Grafana: legge la richiesta e risponde.

Uniform Resource Locator (URL)

Fonte: it.wikipedia.org/wiki/Uniform_Resource_Locator

In giallo i separatori: :// : / ? #



<proto>

il protocollo di livello 7: http o https

finisce con ://

<fqdn>

il nome completo del server: il DNS

lo traduce nell'indirizzo IP (livello 3)

<port>

la porta (livello 4), dopo i :

si omette se è quella standard: 80 o 443

<path>

la risorsa sul server, dopo la prima /

è quello che finisce nei log del web server

<querystring>

parametri per il server, dopo il ?

nome=valore, più parametri uniti da &

<anchor>

un punto della pagina, dopo il #

resta nel browser: non arriva al server

HTTP/1.1: richiesta e risposta

La pagina di Wikipedia sull'URL, e le risposte da riconoscere

richiesta (dal client)

```
GET /wiki/Uniform_Resource_Locator?uselang=it HTTP/1.1
Host: it.wikipedia.org
User-Agent: curl/8.5.0
Accept: */*
```

riga vuota

La riga vuota chiude la richiesta.

#cite_note-2 non c'è: l'anchor resta nel browser.

risposta (dal server)

```
HTTP/1.1 200 OK
Date: Wed, 28 Oct 2026 10:15:02 GMT
Content-Type: text/html; charset=UTF-8
Content-Language: it

<!DOCTYPE html>
<html class="client-nojs" lang="it" dir="ltr">
... (il resto della pagina)
```

riga vuota: poi arriva il body

302 Found: vai altrove

```
$ curl -I http://its.fabersoft.net:3000
HTTP/1.1 302 Found
Location: /login
```

Grafana risponde e rimanda alla pagina di login.

404 Not Found: non esiste

```
GET /wiki/Pagina_che_non_esiste HTTP/1.1
HTTP/1.1 404 Not Found
Content-Type: text/html; charset=UTF-8
```

Il server risponde, ma quel path non c'è.

Qualsiasi codice vuol dire che il livello 7 risponde: 2xx ok, 3xx altrove, 4xx errore del client, 5xx del server.

I log non arrivano?

Un livello alla volta, come nella lezione 1

1. `nc -zv its.fabersoft.net 3100` — livello 4: la porta risponde?
2. `curl http://its.fabersoft.net:3100/ready` — livello 7
3. `docker compose ps` — Alloy gira?
4. `docker compose logs alloy` — cosa dice?

Gli errori di Alloy dicono quasi sempre cosa non va: URL, porta o una virgola.

Strumenti per livello

Dal client si sale da 3 a 7; sul server si guarda chi ascolta e cosa dice

	dal client	sul server
7 applicazione	`curl -I http://host:3000`	`docker compose logs -f grafana` `journalctl -u nome-servizio`
4 porta	`nc -zv host 3000`	`sudo ss -lptn | grep 3000` `sudo tcpdump -ni any port 3000`
3 IP	`ping -c 3 host`	`ip a` `sudo tcpdump -ni any icmp`

Prima di uscire

1. **Cosa fa docker compose up -d?**
2. **Perché è il PC a spedire i log al VPS?**
3. **Come vedo in Grafana i log di un compagno?**

Per la lezione 5

Lasciate la cartella pc-studente com'è:

ad Alloy aggiungiamo le metriche del PC.